

Angular Frontend Developer Assessment — Mini Pokédex

Overview

Build a **Mini Pokédex** — a single-page Angular application where a user can browse Pokémon and build a team, using GraphQL.

Expected effort: ~10 hours. Time limit: 3 days.

The scope is intentionally small. We are not looking for feature quantity — we are looking at **how** you build: project structure, state management, and how you handle loading, empty, and error states. A polished small app beats an unfinished big one.

Required reading: Follow the folder structure, naming conventions, component patterns, and commit standards from the attached `DEVELOPER_GUIDE.md`. Structure and conventions are a major part of the evaluation. (Ignore the AWS Amplify parts — this task uses no auth and the APIs below instead.)

Non-Negotiable Requirement: UI States

Every view that loads async data (Pokédex table, detail panel, team list, autocomplete dropdown) must handle **all four states**:

1. **Loading** — skeleton or spinner (no blank screens, no layout jumps)
2. **Empty** — a meaningful message when there is no data (e.g., search with no results)
3. **Error** — a user-friendly message with a **Retry** action (no raw error dumps, no silent failures)
4. **Success** — the data

We will test this by throttling and cutting the network. Missing states will be treated as failed requirements, not polish.

GraphQL APIs

Public API (Queries)

`https://beta.pokeapi.co/graphql/v1beta` — free, no auth.

```

# Fetch Pokémon list with types and stats
query GetPokemon($limit: Int, $offset: Int) {
  pokemon_v2_pokemon(limit: $limit, offset: $offset) {
    id
    name
    height
    weight
    pokemon_v2_pokemontypes {
      pokemon_v2_type {
        name
      }
    }
    pokemon_v2_pokemonstats {
      base_stat
      pokemon_v2_stat {
        name
      }
    }
    pokemon_v2_pokemonsprites {
      sprites
    }
  }
}

# Fetch abilities for one Pokémon
query GetAbilities($pokemonId: Int) {
  pokemon_v2_pokemonability(where: { pokemon_id: { _eq: $pokemonId } }) {
    pokemon_v2_ability {
      name
      pokemon_v2_abilityeffecttexts(where: { language_id: { _eq: 9 } }) {
        short_effect
      }
    }
    is_hidden
  }
}

```

Local Mock Server (Mutations)

Run with: `npx json-graphql-server db.js --port 4000`

```
// db.js
module.exports = {
  trainers: [
    {
      id: 1,
      name: "Ash Ketchum",
      region: "Kanto",
      avatar_url:
        "https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/trainers/1.png",
    },
    {
      id: 2,
      name: "Misty Waterflower",
      region: "Kanto",
      avatar_url: "",
    },
  ],
  teams: [
    {
      id: 1,
      trainer_id: 1,
      name: "Kanto Starters",
      pokemon_ids: [25, 6, 9, 3, 131, 143],
      created_at: "2024-01-15T10:00:00Z",
    },
    {
      id: 2,
      trainer_id: 1,
      name: "Johto Squad",
      pokemon_ids: [157, 181, 214, 248, 197, 169],
      created_at: "2024-03-20T14:30:00Z",
    },
    {
      id: 3,
      trainer_id: 2,
      name: "Water Specialists",
      pokemon_ids: [121, 130, 134, 73, 91, 99],
      created_at: "2024-02-10T09:00:00Z",
    },
  ],
};
```

Required Tasks

1. GraphQL Integration

Queries (PokéAPI):

- Paginated Pokémon list with types, stats, and sprites
- Single Pokémon details by ID (stats + abilities)

Queries & Mutations (Local mock):

- Fetch teams
- Create a team / delete a team

Resilience:

- `retry` with delay on PokéAPI calls
- Every call must surface errors to the UI (see UI States above)

2. RxJS State Management

Build a **custom store** (no NgRx/Akita/NgXS):

```
src/app/pokedex/state/
├── pokemon.store.ts      // BehaviorSubject-based store, caches fetched Pokémon
└── pokemon.selectors.ts // Derived observables (filtered, sorted, paged)
src/app/teams/state/
└── team.store.ts        // Teams + create/delete with optimistic updates
```

Requirements:

- `BehaviorSubject` as the core state holder; selectors as derived streams using `map`, `distinctUntilChanged`, `combineLatest`
- **Pokémon search:** `debounceTime(300)` → `distinctUntilChanged()` → `switchMap`
- **Optimistic update:** when creating a team, show it in the list immediately; roll back and show an error toast/message if the mutation fails
- `shareReplay(1)` where a derived stream is consumed by multiple subscribers
- No subscription leaks — `takeUntilDestroyed()` / `DestroyRef` / `async` pipe

3. Angular Signals

- `signal()` for UI state (selected Pokémon, panel open/closed, loading flags)
- `computed()` for derived values (e.g., team type distribution, team total base stats)
- `effect()` to persist the selected team to `localStorage`
- `toSignal()` to bridge store selectors into templates
- `input()` / `output()` for all component communication — no legacy decorators
- All components: standalone, `ChangeDetectionStrategy.OnPush`, `inject()`

4. Pokédex Table

- Columns: sprite, name, types (colored badges), HP, Attack, Defense, Sp.Atk, Sp.Def, Speed, total
- **Sortable** by any stat column
- **Client-side pagination:** 10 / 25 / 50 per page
- **Filters:** debounced text search on name + dropdown filter on type
- **Row click** opens a detail side panel (slide-in) with full info and a **radar chart** of the 6 base stats (ngx-charts, Chart.js, or ECharts — chart must animate when a different Pokémon is selected)

5. Team Builder Form (Reactive Forms)

- **Team name:** required, 3–30 chars, **async validator** — name must be unique against existing teams (debounced)
- **Pokémon picker:** autocomplete input (typeahead against the cached list or the API, debounced); selected Pokémon shown as removable chips; min 1, max 6
- Inline validation errors respecting `dirty / touched` state — no errors on pristine fields, no `alert()`
- On submit: create via mutation with the optimistic flow from Task 2

6. Code Quality

- Folder structure, file naming, and component/service patterns per `DEVELOPER_GUIDE.md`
 - JSDoc comments on all **public methods of stores and services** (components: only where non-obvious)
 - Conventional commits (`feat(pokedex): ...`) per the guide — commit as you go, not one giant commit
-

Bonus (optional — pick at most ONE, only if under 10 hours)

1. **Virtual scroll** Pokédex with `@angular/cdk` instead of pagination, with skeleton rows
2. **Drag-and-drop** Pokémon from the table into team slots with `@angular/cdk/drag-drop`
3. **Type highlight directive** `[appTypeHighlight]` that marks table rows strong/weak vs. a selected type
4. **Skeleton shimmer + micro-animations:** staggered card entry, toast notifications for mutations

A missing bonus costs nothing. A broken bonus, or missing required states because time went into a bonus, does.

Evaluation Criteria

Category	Weight	What We Look For
UI States & Error Handling	25%	Loading / empty / error / success handled everywhere, retry works, no blank UI
Project Structure & Conventions	20%	Folder structure, naming, commits per DEVELOPER_GUIDE.md
RxJS State Management	20%	Operator usage, optimistic update + rollback, no leaks
Signals	15%	Correct signal/computed/effect/toSignal usage, OnPush everywhere
UI Components	10%	Table features, form validation UX, chart reactivity
Tests & README	10%	Tests pass and are meaningful, README is accurate

Deliverables

1. GitHub repository with clean conventional-commit history
2. `README.md` : setup instructions (including the mock server), short architecture note, what you'd improve with more time
3. Minimum **3 unit tests**: one store method (e.g., optimistic rollback), one selector or `computed` , one form validator
4. App must build and run with `ng serve` without errors (mock server on port 4000)

Time limit: 3 days from receiving this task.